

Transformers from the Matrices Up

The Transformer, published in 2017, is the architecture inside BERT, GPT, Llama and nearly every language model you will ever use. This guide builds it one piece at a time, with numbers small enough to check on a calculator, until you can write it yourself and explain every line.

TIME	YOU NEED	YOU WILL BUILD
1 week, about 3 hours a day	Phase 1: embeddings, softmax, the RNN bottleneck	Self-attention from scratch, then a small GPT trained on your own text

HOW TO USE THIS GUIDE

This is the phase most candidates skip, which is exactly why it separates you. Section 3 is the core: do the worked example with a pen before reading the code. Watch the two 3Blue1Brown videos in Resources on day one — they make everything here visual.

Section 2 is a short matrix refresher. If you are comfortable multiplying matrices and reading shapes, skip it.

INSIDE THIS GUIDE

1. Why transformers exist
2. Matrix multiplication refresher
3. Self-attention, step by step
4. Multi-head attention
5. Positional information
6. Inside a transformer block
7. Masking
8. The three model families
9. How text gets generated
10. Cost and the tricks that fight it
11. Project: attention, then a tiny GPT
12. Interview answers
13. Resources, glossary, checklist

1 Why transformers exist

Phase 1 ended with two problems of RNNs. They read one token at a time, so they can't use a GPU's ability to do thousands of calculations at once, and information from early tokens fades over long sequences.

The 2017 paper *Attention Is All You Need* removed recurrence completely. In a Transformer, **every token looks directly at every other token in a single step**. A connection between word 1 and word 500 is exactly as easy to learn as one between neighbours, and all positions are processed in parallel.

THE WHOLE MODEL AT A GLANCE (DECODER-ONLY, LIKE GPT)



Each block contains two parts: **attention**, where tokens share information with each other, and a **feed-forward network**, where each token processes what it gathered. GPT-2 small stacks 12 such blocks with 768-number vectors, for about 124 million parameters in total.

2 Matrix multiplication refresher

To multiply matrix A by matrix B, each entry of the result is the dot product of a *row* of A with a *column* of B.

THE SHAPE RULE

$(a \times b)$ multiplied by $(b \times c)$ gives $(a \times c)$. The inner numbers must match and disappear. A (2×3) times a (3×2) is a (2×2) . A (2×3) times a (2×3) is an error.

WORKED EXAMPLE

$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$ (shape 2×3) $B = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$ (shape 3×2)

Row 1 of A with column 1 of B: $1 \times 1 + 2 \times 0 + 3 \times 1 = 4$. Row 1 with column 2: $1 \times 0 + 2 \times 1 + 3 \times 1 = 5$.

Row 2 with column 1: $4 + 0 + 6 = 10$. Row 2 with column 2: $0 + 5 + 6 = 11$.

$A \times B = \begin{bmatrix} 4 & 5 \\ 10 & 11 \end{bmatrix}$, shape 2×2 .

The **transpose** of a matrix, written A^T , flips rows into columns: a (2×3) becomes a (3×2) . In PyTorch you will see `A @ B` for multiplication and `A.transpose(-2, -1)` for transpose.

3 Self-attention, step by step

3.1 The intuition

“The animal didn’t cross the street because **it** was too tired.” What does “it” mean? To build a good representation of “it”, the model should pull in information from “animal”. Change “tired” to “wide” and “it” should pull from “street” instead.

Self-attention computes, for every token, how much to borrow from every other token, and then mixes their information in those proportions. It is “self” attention because the sentence attends to itself.

3.2 Query, Key, Value

Each token's embedding x is multiplied by three different learned weight matrices to produce three new vectors:

- **Query** $q = x \cdot W_Q$ — “what am I looking for?”
- **Key** $k = x \cdot W_K$ — “what do I contain, as a label others can match?”
- **Value** $v = x \cdot W_V$ — “what information will I actually hand over?”

THINK OF IT LIKE

Searching a library. Your *query* is what you type into the search box. Each book has a *key*: the title and tags on its catalogue card. You compare your query against every key to get relevance scores. Then you take the *values* — the books' actual contents — and read mostly from the relevant ones and barely at all from the others.

3.3 The formula

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T \div \sqrt{d_k}) V$$

Stack all tokens' queries as rows of a matrix Q , and likewise K and V . With n tokens and vectors of size d_k :

1. **Scores** = $Q K^T$, shape $(n \times n)$. Row i , column j is how well token i 's query matches token j 's key.
2. **Scale**: divide every score by $\sqrt{d_k}$. (Section 3.5 explains why.)
3. **Softmax** each row, so each token's attention weights are positive and add to 1.
4. **Mix**: multiply the weights by V . Each token's output is a weighted blend of all tokens' values.

3.4 A worked example you can check by hand

Three tokens, $d_k = 2$. To keep the arithmetic small, pretend the learned matrices are identity matrices, so Q, K and V all equal the embeddings X :

$$X = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}$$

STEP 1 — SCORES = $Q K^T = X X^T$

Row 1: $[1 \cdot 1 + 0 \cdot 0, 1 \cdot 0 + 0 \cdot 1, 1 \cdot 1 + 0 \cdot 1] = [1, 0, 1]$

Row 2: $[0, 1, 1]$ Row 3: $[1, 1, 2]$

STEP 2 — DIVIDE BY $\sqrt{2} = 1.414$

Row 1: $[0.707, 0, 0.707]$ Row 2: $[0, 0.707, 0.707]$ Row 3: $[0.707, 0.707, 1.414]$

STEP 3 — SOFTMAX EACH ROW

Row 1: $e^{0.707} = 2.028, e^0 = 1, e^{0.707} = 2.028$. Total 5.056. Weights = **[0.401, 0.198, 0.401]**.

Row 3: $2.028, 2.028, e^{1.414} = 4.113$. Total 8.169. Weights = **[0.248, 0.248, 0.504]**.

STEP 4 — MIX THE VALUES

Output 1 = $0.401 \cdot [1, 0] + 0.198 \cdot [0, 1] + 0.401 \cdot [1, 1] = \mathbf{[0.802, 0.599]}$

Output 3 = $0.248 \cdot [1, 0] + 0.248 \cdot [0, 1] + 0.504 \cdot [1, 1] = \mathbf{[0.752, 0.752]}$

Read what happened. Token 3, $[1, 1]$, matched itself most strongly (0.504) but still borrowed from both others. Token 1 ignored token 2 the most, because their vectors point in unrelated directions. Every output is now a blend informed by context — which is exactly what “it” needed from “animal”.

3.5 Why divide by $\sqrt{d_k}$?

The dot product of two random vectors grows with their length: with $d_k = 64$ or 128 , raw scores can easily be 10 or 30. Softmax on large numbers becomes extreme.

SCORES	SOFTMAX RESULT	WHAT IT MEANS
$[1, 2, 3]$	$[0.090, 0.245, 0.665]$	A healthy spread; every token contributes.
$[10, 20, 30]$	$[0.000, 0.000, 1.000]$	Almost one-hot. The other tokens are ignored completely.

When softmax is that saturated, small changes to the scores barely change the output, so the gradients are nearly zero and learning stalls. Dividing by $\sqrt{d_k}$ keeps the scores in a sensible range whatever the vector size.

3.6 Self-attention in code

```
import math
import torch

def my_attention(q, k, v, causal: bool = False):
    d_k = q.shape[-1]
    scores = q @ k.transpose(-2, -1)          # (... , seq_len, seq_len)
    scores = scores / math.sqrt(d_k)
    if causal:
        seq_len = q.shape[-2]
        mask = torch.tril(torch.ones(seq_len, seq_len))
        scores = scores.masked_fill(mask == 0, float("-inf"))
    weights = torch.softmax(scores, dim=-1)    # each row sums to 1
    return weights @ v                        # (... , seq_len, d_k)
```

TRY IT YOURSELF

Call `my_attention` with `X = torch.tensor([[1., 0.], [0., 1.], [1., 1.]])` passed as `q`, `k` and `v`. You should get the numbers from Section 3.4. Then change `causal` to `True` and predict the first row of output before running it. (Answer: token 1 can only see itself, so its output is exactly `[1, 0]`.)

4 Multi-head attention

One attention computation learns one way of relating tokens. Language has many relationships at once: who did what to whom, which adjective describes which noun, what a pronoun refers to, which words are simply nearby. **Multi-head attention** runs several attention computations in parallel, each with its own W_Q , W_K , W_V , so each *head* can specialise. (In practice what heads learn is messier than this tidy story, but the benefit is real.)

The heads don't make the model bigger. The model dimension is split between them:

$$\text{head_dim} = \text{d_model} \div \text{n_heads} \quad \text{e.g. } 768 \div 12 = 64$$

Each head attends using 64-number vectors; the 12 outputs are concatenated back into 768 numbers and passed through one more linear layer, W_O .

```
import torch.nn as nn

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        assert d_model % n_heads == 0
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads
        self.W_q = nn.Linear(d_model, d_model, bias=False)
        self.W_k = nn.Linear(d_model, d_model, bias=False)
        self.W_v = nn.Linear(d_model, d_model, bias=False)
        self.W_o = nn.Linear(d_model, d_model, bias=False)

    def split_heads(self, x):
        batch, seq_len, d_model = x.shape
        x = x.view(batch, seq_len, self.n_heads, self.head_dim)
        return x.transpose(1, 2)  # (batch, n_heads, seq_len, head_dim)

    def forward(self, x, causal: bool = True):
        q = self.split_heads(self.W_q(x))
        k = self.split_heads(self.W_k(x))
        v = self.split_heads(self.W_v(x))
        out = my_attention(q, k, v, causal)  # (batch, n_heads, seq_len, head_dim)
        batch, n_heads, seq_len, head_dim = out.shape
        out = out.transpose(1, 2).contiguous().view(batch, seq_len, n_heads * head_dim)
        return self.W_o(out)  # (batch, seq_len, d_model)
```

The only new trick is reshaping: `view` cuts each 768-number vector into 12 pieces of 64, and `transpose` moves the head dimension forward so attention runs on every head at once.

5 Positional information

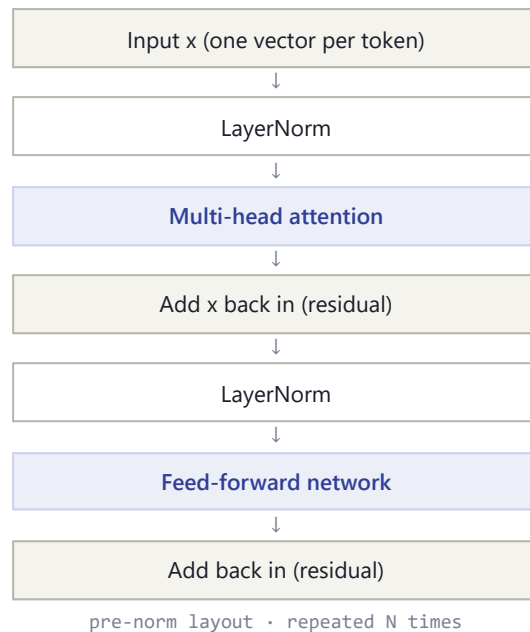
Attention on its own has **no sense of order**. Shuffle the words of a sentence and every token computes the same set of scores against the same set of other tokens. “Dog bites man” and “man bites dog” would look identical. So position must be injected somehow.

METHOD	HOW IT WORKS	USED BY	TRADE-OFF
Sinusoidal	Add a fixed pattern of sine and cosine waves of different frequencies to each position’s embedding.	Original Transformer	No training needed; works poorly far beyond trained lengths.
Learned	A trainable vector for each position (position 0, 1, 2...) added to the embedding.	BERT, GPT-2	Simple, but no vectors exist beyond the maximum trained length (512 for BERT, 1,024 for GPT-2).
RoPE (rotary)	Rotate each query and key vector by an angle proportional to its position. The dot product between two tokens then depends on their <i>distance apart</i> .	Llama, Mistral, Qwen and most modern open models	Captures relative position naturally; can be stretched to longer contexts with scaling tricks.
ALiBi	No position vectors at all. Subtract a penalty from each attention score that grows with the distance between the two tokens.	BLOOM, MPT	Extends gracefully to inputs longer than training.

THINK OF IT LIKE

Sinusoidal encoding is like the hands of a clock. The second hand spins fast, the minute hand slower, the hour hand slowest; together their positions uniquely name every moment. RoPE is like describing where two people stand by the *angle between them* rather than their street addresses — what matters is how far apart they are.

6 Inside a transformer block



6.1 Residual connections

Instead of $\text{output} = \text{layer}(x)$, a block computes $\text{output} = x + \text{layer}(x)$. The layer only has to learn the *change* to make, not rebuild the whole representation. It also gives gradients a direct highway backwards through dozens of blocks, which is what makes very deep models trainable.

THINK OF IT LIKE

Editing a document with tracked changes rather than retyping it from scratch at every review. If a reviewer has nothing useful to add, the document simply passes through unchanged.

6.2 LayerNorm and RMSNorm

LayerNorm rescales each token's vector to have mean 0 and variance 1, then applies a learned scale and shift. This keeps numbers in a stable range as they flow through many layers. **RMSNorm** skips subtracting the mean and just divides by the root-mean-square. It is cheaper and works as well, so Llama and most recent models use it.

6.3 Pre-norm versus post-norm

The original paper normalised *after* adding the residual (post-norm). Modern models normalise *before* each sub-layer (pre-norm, as drawn above). Pre-norm trains much more stably in deep models and needs less careful tuning.

6.4 The feed-forward network

Applied to each token independently: expand the vector to $4\times$ its size ($768 \rightarrow 3,072$), apply a non-linearity, shrink it back ($3,072 \rightarrow 768$). A useful mental model: *attention is where tokens talk to each other; the feed-forward network is where each token thinks on its own*. Research suggests much of a model's stored factual knowledge lives in these layers.

The non-linearity: **ReLU** sets negatives to zero; **GELU** is a smooth version (BERT, GPT-2); **SwiGLU** is a gated variant that most recent models use.

```
class Block(nn.Module):
    def __init__(self, d_model: int, n_heads: int):
        super().__init__()
        self.norm1 = nn.LayerNorm(d_model)
        self.attn = MultiHeadAttention(d_model, n_heads)
        self.norm2 = nn.LayerNorm(d_model)
        self.ffn = nn.Sequential(
            nn.Linear(d_model, 4 * d_model),
            nn.GELU(),
            nn.Linear(4 * d_model, d_model),
        )

    def forward(self, x):
        x = x + self.attn(self.norm1(x))    # tokens talk to each other
        x = x + self.ffn(self.norm2(x))    # each token thinks alone
        return x
```

7 Masking

7.1 The causal mask

A GPT-style model is trained to predict the next token. When it is working on position 3, it must not be able to see positions 4, 5 and beyond — that would be copying the answer. The **causal mask** is a lower-triangular matrix: 1 means “may look”, 0 means “blocked”.

query ↓ / key →	The	cat	sat	down
The	1	0	0	0
cat	1	1	0	0
sat	1	1	1	0
down	1	1	1	1

Before softmax, every blocked score is set to $-\infty$. Since $e^{-\infty} = 0$, blocked positions receive exactly zero weight. In code: `torch.tril(torch.ones(4, 4))`.

7.2 The padding mask

Sentences in a batch have different lengths, so shorter ones are filled with `[PAD]` tokens. Nobody should attend to padding. The padding mask blocks the padding *columns* for every row.

query ↓ / key →	I	love	NLP	[PAD]	[PAD]
I	1	1	1	0	0
love	1	1	1	0	0
NLP	1	1	1	0	0

This shows a non-causal model like BERT. In a GPT-style model with padding, both masks apply together.

REMEMBER

Causal mask: blocks the *future*, triangle-shaped, needed for generation. Padding mask: blocks *filler tokens*, column-shaped, needed for batching. Interviewers genuinely ask you to draw both.

8 The three model families

FAMILY	EXAMPLES	WHAT EACH TOKEN SEES	TRAINING OBJECTIVE	BEST FOR
Encoder-only	BERT, RoBERTa	The whole sentence, both directions.	Masked language modelling: hide about 15% of tokens and predict them.	Classification, NER, producing embeddings for search.
Decoder-only	GPT, Llama, Mistral, most chat models	Only tokens to its left (causal mask).	Next-token prediction.	Generating text; essentially all modern LLMs.
Encoder-decoder	T5, BART	Encoder sees all of the input; decoder sees earlier outputs plus the encoder.	Reconstructing corrupted text; text-to-text tasks.	Translation, summarization.

Encoder-decoder models add **cross-attention** in the decoder: queries come from the decoder's tokens, while keys and values come from the encoder's output. That is how the decoder "looks at" the input sentence — the modern form of the Bahdanau attention from Phase 1.

BERT also solves Phase 1's "bank" problem. Because every token's output vector has mixed in its context through attention, "bank" next to "river" and "bank" next to "loan" come out as different vectors. These are **contextual embeddings**.

9 How text gets generated

1. Run the prompt through the model. At the last position, the final layer outputs a **logit** for every token in the vocabulary (often 50,000 to 200,000 of them).
2. Softmax turns the logits into probabilities.
3. Pick one token from those probabilities.
4. Append it to the input and repeat from step 1, until an end-of-text token or a length limit.

This is called **autoregressive** generation: each output becomes part of the next input. *How* to pick in step 3 — greedy, temperature, top-p — is covered in Phase 3.

10 Cost and the tricks that fight it

10.1 Attention is $O(n^2)$

The score table is $n \times n$. For 1,000 tokens that is 1 million entries — per head, per layer. For 100,000 tokens it is 10 billion. **Doubling the input length quadruples the attention cost.** This is why long context windows are expensive and why so much research targets attention efficiency.

10.2 The KV cache

When generating token 101, the keys and values for tokens 1–100 are exactly the same as they were when generating token 100. Recomputing them is wasteful, so the model stores them and computes q , k and v only for the one new token. That store is the **KV cache**. It makes generation dramatically faster, at the cost of memory.

$$\text{KV cache bytes} = 2 \times \text{layers} \times \text{KV heads} \times \text{head_dim} \times \text{sequence length} \times \text{batch size} \times \text{bytes per number}$$

The leading 2 is because both a key and a value are stored.

WORKED EXAMPLE: A 7B MODEL WITH LLAMA 2 7B'S SHAPE

32 layers, 32 KV heads, head_dim 128, one conversation of 4,096 tokens, 16-bit numbers (2 bytes):

$2 \times 32 \times 32 \times 128 \times 4,096 \times 1 \times 2 = 2,147,483,648$ bytes $\approx$ **2 GB** for a single 4K-token conversation, on top of the $\sim$ 14 GB of model weights.

10.3 MQA and GQA: shrinking the cache

VARIANT	HOW KEYS AND VALUES ARE SHARED	CACHE SIZE (32 QUERY HEADS)
Multi-Head Attention (MHA)	Every head has its own keys and values.	32 KV heads — 2 GB in the example
Grouped-Query Attention (GQA)	Query heads are grouped; each group shares one set of keys and values.	e.g. 8 KV heads — 4× smaller, 512 MB
Multi-Query Attention (MQA)	All heads share a single set of keys and values.	1 KV head — 32× smaller, small quality cost

GQA is the common modern compromise: most of MQA's memory savings, most of MHA's quality.

10.4 FlashAttention

FlashAttention computes *exactly the same result* as normal attention. It is not an approximation. It processes attention in blocks small enough to fit in the GPU's tiny but very fast on-chip memory and never stores the full $n \times n$ table. Much faster and far less memory, with identical output — an engineering win, not a modelling change.

10.5 Sliding-window attention

Each token attends only to the previous W tokens (say 4,096) instead of all of them, so cost grows linearly with length. Information from further back still flows forward indirectly through the stacked layers. Mistral 7B used this.

REMEMBER

$O(n^2)$ is the problem. KV cache speeds up generation but costs memory. GQA/MQA shrink that memory. FlashAttention is exact and faster. Sliding windows trade some long-range precision for linear cost.

11 Project: attention, then a tiny GPT

Part A — prove your attention is correct

PyTorch ships an official, optimised attention function. If yours matches it, your implementation is right.

```
import torch
import torch.nn.functional as F

q = torch.randn(1, 5, 16)
k = torch.randn(1, 5, 16)
v = torch.randn(1, 5, 16)

mine = my_attention(q, k, v, causal=True)
reference = F.scaled_dot_product_attention(q, k, v, is_causal=True)

print(torch.allclose(mine, reference, atol=1e-5))    # should print True
```

Then check that `MultiHeadAttention` and `Block` produce outputs of shape `(batch, seq_len, d_model)` for a random input of shape `(2, 10, 64)` with 8 heads. Commit all three pieces with a short README explaining each shape.

Part B — build GPT with Karpathy, then train your own

1. **Watch “Let’s build GPT: from scratch, in code, spelled out”** and type along. The video is about two hours; budget five or six with pauses. It builds a character-level GPT on Shakespeare using exactly the pieces in this guide.
2. **Clone nanoGPT** and run the Shakespeare character example from its README:

```
python data/shakespeare_char/prepare.py
python train.py config/train_shakespeare_char.py
python sample.py --out_dir=out-shakespeare-char
```

Use a Colab GPU. On a laptop CPU, the README gives a smaller command that finishes in a few minutes:

```
python train.py config/train_shakespeare_char.py --device=cpu --compile=False --eval_iters=20 --
log_interval=1 --block_size=64 --batch_size=12 --n_layer=4 --n_head=4 --n_embd=128 --
max_iters=2000 --lr_decay_iters=2000 --dropout=0.0
```

3. **Train on text you chose.** Copy the `data/shakespeare_char` folder to `data/my_text`, put your own `input.txt` inside (song lyrics, a public-domain novel, your old essays — at least about 1 MB of text), run its `prepare.py`, and point a copied config’s `dataset` and `out_dir` at it.
4. **Save samples along the way.** Generate text at around 250, 1,000 and the final iteration, and keep all three, including the early garbage. Showing the progression from noise to recognisable style is a far better demo

than a single good sample.

A NOTE ON NANO GPT

Karpathy's README now marks nanoGPT as old and deprecated, and points to his newer project **nanochat**. For *learning* this week, nanoGPT is still the right choice: it is small, it matches the video line for line, and its Shakespeare example runs on free hardware. Look at nanochat afterwards to see what a complete modern training pipeline adds.

12 Interview answers

Q Write self-attention on the board.

Project the input X into queries, keys and values with three learned matrices. Compute scores as Q times K -transpose, divide by the square root of d_k , apply a causal mask if generating, softmax each row, and multiply the resulting weights by V . Every output row is a weighted mixture of all value vectors. Then state the shapes: n by d in, n by n weights, n by d_k out.

Q Why scale by $\sqrt{d_k}$? What breaks without it?

Dot products of random vectors have variance proportional to their dimension, so with large d_k the scores become large. Softmax on large scores saturates to nearly one-hot, where gradients are almost zero and training stalls. Dividing by the square root of d_k keeps the variance around one.

Q What exactly does the KV cache store, and how large does it get?

The key and value vectors for every previous token, at every layer, so each new token only computes its own. Size is 2 times layers times KV heads times head dimension times sequence length times batch times bytes per number — about 2 GB for a 7B model with 32 full KV heads at 4K tokens in 16-bit. It grows linearly with context length and batch size, which is why GQA is used to shrink it.

Q BERT or GPT for this task, and why?

For understanding tasks where the whole input is available — classification, NER, embeddings for retrieval — an encoder like BERT, because it reads context in both directions and is small and fast. For anything that must produce text, a decoder like GPT. In practice, large decoder models can also do classification through prompting, so the choice often comes down to cost and latency.

Q Why do transformers train faster than RNNs?

During training, all positions are processed in parallel with matrix multiplications that GPUs are built for, while an RNN must process position t before $t+1$. Attention also connects distant tokens in one step, avoiding vanishing gradients over long paths.

Q What are residual connections for?

They add each sub-layer's input to its output, so a layer learns a correction rather than a full transformation, and gradients can flow straight back through many layers. Without them, very deep stacks are extremely hard to train.

Q Is FlashAttention an approximation?

No. It produces the same output as standard attention. It restructures the computation into blocks that fit in fast GPU memory and avoids materialising the full n -by- n matrix, which saves memory and time without changing the result.

13 Resources, in order

START HERE *day one*

Transformers, the tech behind LLMs — 3Blue1Brown, Chapter 5

Video · ~27 min

Embeddings, the overall data flow and what the model outputs, animated beautifully.

youtube.com/watch?v=wjZofJX0v4M

Attention in transformers, step-by-step — 3Blue1Brown, Chapter 6

Video · ~26 min

Queries, keys, values, masking and multiple heads. Watch it right after Chapter 5 and again after Section 4.

youtube.com/watch?v=eMlx5FFNoYc

The Illustrated Transformer — Jay Alammar

Article · ~1 h

The most widely recommended visual walkthrough of the original architecture, including encoder-decoder attention.

jalammar.github.io/illustrated-transformer/

Transformer Explainer

Interactive tool

A live GPT-2 running in your browser. Type a prompt and inspect the attention weights and probabilities at every stage.

poloclub.github.io/transformer-explainer/

NEXT

Let's build GPT: from scratch, in code, spelled out — Andrej Karpathy

Video · ~2 h (plan 5-6 h)

The project's main resource. Type every line yourself.

youtube.com/watch?v=kCc8FmEb1nY

nanoGPT — Karpathy

Code repository

The small, readable GPT training code used in Part B. `model.py` is about 300 lines — read it after the video.

github.com/karpathy/nanoGPT

The Illustrated BERT — Jay Alammar

Article · ~40 min

How encoder-only models are pre-trained and fine-tuned, and where contextual embeddings come from.

jalammar.github.io/illustrated-bert/

LLM Visualization — Brendan Bycroft

Interactive 3D tool

Walk through every matrix of a small GPT in 3D, step by step. Excellent for making the shapes concrete.

bbycroft.net/llm

How might LLMs store facts — 3Blue1Brown, Chapter 7

Video · ~23 min

What the feed-forward layers might be doing, following on from Section 6.4.

youtube.com/watch?v=9-Jl0dxWQs8

GO DEEPER

Attention Is All You Need (Vaswani et al., 2017)

Paper · read after the illustrated guides

The original paper. After this guide, Section 3 of the paper (model architecture) will read clearly.

arxiv.org/abs/1706.03762

The Annotated Transformer — Harvard NLP

Article with code · ~3 h

The paper printed line by line alongside a working PyTorch implementation.

nlp.seas.harvard.edu/annotated-transformer/

LLMs-from-scratch — Sebastian Raschka

Code for the book

Chapters on working with text data, coding attention mechanisms and implementing a GPT model, as clear notebooks.

github.com/rasbt/LLMs-from-scratch

Neural Networks: Zero to Hero — Karpathy

Video series

The full series that leads up to the GPT video, if you want every foundation built by hand.

karpathy.ai/zero-to-hero.html

nanochat — Karpathy

Code repository

The newer, complete pipeline that nanoGPT's README now points to. Explore it once Part B is done.

github.com/karpathy/nanochat

RoFormer (RoPE) and FlashAttention papers

Papers · skim

The original sources for rotary position embeddings and exact, memory-efficient attention.

arxiv.org/abs/2104.09864 · arxiv.org/abs/2205.14135

14 Glossary

Transformer

Architecture built from attention and feed-forward blocks, with no recurrence.

Self-attention

Each token builds its output as a weighted mix of all tokens' values.

Query / key / value

What a token looks for / how it is matched / what it passes on.

d_model / d_k

Size of each token's vector / size of query and key vectors.

Head

One of several parallel attention computations.

Positional encoding

Information added so the model knows token order.

RoPE

Rotary position embedding: rotates queries and keys by position.

Residual connection

Adding a layer's input to its output.

LayerNorm / RMSNorm

Rescaling each vector to a stable range.

Feed-forward network

Per-token expand, non-linearity, shrink.

Causal mask

Stops tokens seeing future positions.

Padding mask

Stops tokens attending to filler padding.

Cross-attention

Decoder queries attending to encoder keys and values.

Contextual embedding

A token vector that depends on the surrounding words.

Masked language modelling

BERT's training: hide tokens and predict them.

Autoregressive

Generating one token at a time, feeding each back in.

KV cache

Stored keys and values from earlier tokens during generation.

GQA / MQA

Sharing keys and values across groups of heads / all heads.

FlashAttention

Exact attention computed in memory-efficient blocks.

Sliding-window attention

Each token attends only to a fixed number of recent tokens.

15 You are done with Phase 2 when...

- ☐ You can multiply two small matrices by hand and predict any result's shape.
- ☐ You can do the Section 3.4 attention example on paper without looking.
- ☐ You can explain Q, K and V with an analogy and the $\sqrt{d_k}$ scaling with the softmax table.
- ☐ Your `my_attention` matches `scaled_dot_product_attention` (`allclose` prints True).
- ☐ You can draw a transformer block, and the causal and padding masks, from memory.
- ☐ You can compare encoder-only, decoder-only and encoder-decoder models in one sentence each.
- ☐ You can compute the KV cache size for a given model and explain how GQA shrinks it.
- ☐ You have trained nanoGPT on text you chose and kept samples from early, middle and late training.

Next: Phase 3 — LLM Behaviour and Control. You will see how a pre-trained transformer becomes an assistant, and how to steer, measure and cheaply adapt it.